

# 实验05 – 禁止循环

**主题：** 如果你循环，你就输了。

**目标：** 使用向量化构建高频交易（HFT）回测器。

---

## 1. 利害关系（微秒战争）

你现在是一家对冲基金的量化分析师。

你的老板交给你10年的逐笔数据（数百万行）。

你需要测试一个交易策略，看看它是否赚钱。

- **初级方法：** 遍历每一天，检查价格，决定买入/卖出。（时间：20分钟）。
- **高级方法：** 在整个内存块上同时向量化数学操作。（时间：0.2秒）。

**今天的规则：**

你**禁止**编写 `for` 循环或 `while` 循环来处理逻辑。

如果你使用 `df.iterrows()`，你实验不及格。

---

## 第1部分：设置（市场生成器）

我们需要一个波动的股票市场。我们将生成100万分钟的交易日数据。

**运行此代码片段：**

```
import pandas as pd
import numpy as np
import time

# 生成合成股票数据
np.random.seed(42)
rows = 1_000_000

print(f"开盘... 生成 {rows} 分钟的数据。")

df = pd.DataFrame({
    'timestamp': pd.date_range(start='2020-01-01', periods=rows,
    freq='min'),
    'price': 100 + np.cumsum(np.random.normal(0, 1, rows)), # 随机游走
    'volume': np.random.randint(100, 10000, rows)
```

```
} )
```

```
print("市场开盘。当前价格: ", df['price'].iloc[-1])
```

---

## 第2部分: "羞耻循环" (对照组)

我们将实现一个简单的策略:

**策略: "动量买入"**

如果价格比**1分钟前**高, 买入 ('1')。否则, 卖出 ('0')。

**任务:** 使用标准 Python For 循环实现。我们需要衡量这有多痛苦。

```
%%time
# ----- 警告: 以下是不好的代码 -----

# 创建存储信号的地方
signals = []

# 逐行迭代 ("初级"方式)
# 我们从1开始, 因为需要回头看 [i-1]
for i in range(1, len(df)):
    current_price = df['price'].iloc[i]
    prev_price = df['price'].iloc[i-1]

    if current_price > prev_price:
        signals.append(1) # 买入
    else:
        signals.append(0) # 卖出

# 填充第一分钟
signals.insert(0, 0)
df['signal_loop'] = signals
```

**观察:**

根据你的CPU, 这将需要 **10到30秒**。

在HFT世界中, 10秒是永恒。你刚刚损失了数百万美元。

---

## 第3部分: 向量化之神 (Pandas速度)

现在我们做完全相同的逻辑，但我们将列视为**向量**（数学对象），而不是项目列表。

### 概念：

不是看第 `i` 行和 `i-1` 行，我们取**整个价格列**并创建一个向下移动1的克隆。

```
%%time
# ----- 专业代码 -----

# 1. 创建一个移位向量
df['prev_price'] = df['price'].shift(1)

# 2. 直接比较数组（布尔掩码）
# CPU使用SIMD指令并行比较100万个数字。
df['signal_vector'] = (df['price'] > df['prev_price']).astype(int)
```

### 审计：

查看 `%%time` 输出。

它可能需要 **~15毫秒**。

**这是1000倍的加速。**

### 教训：

- 循环发生在**Python**中（慢解释器）。
- 向量化发生在**C**中（编译机器码）。
- 如果你能写成 `列A > 列B`，永远不要写循环。

---

## 第4部分：复杂策略（移动平均线）

真实策略更难。让我们尝试一个**金叉**。

**策略：** 如果价格 > 50窗口移动平均线，买入。

### 任务：

计算50周期移动平均线和信号，**不使用循环**。

```
start_time = time.time()

# 1. 计算滚动平均
df['SMA_50'] = df['price'].rolling(window=50).mean()

# 2. 使用 NumPy "Where" 生成逻辑（类似Excel的IF）
# 语法: np.where(条件, 如果真, 如果假)
```

```
df['signal_sma'] = np.where(df['price'] > df['SMA_50'], 1, 0)

print(f"策略分析完成。时间: {time.time() - start_time:.4f} 秒")
```

### 结果:

你刚刚在眨眼间计算了百万行的复杂统计。

---

## 第5部分: "方法链式" (整洁代码)

行业代码需要可读性。

如果不需要永久保留, 我们避免创建像 `df['prev_price']` 或 `df['SMA_50']` 这样的中间变量。

### 任务:

使用链式 (`.assign()`) 重写上面的整个管道。这是高级Pandas工程师的标志。

```
# "流畅"接口
df_final = (df
    .assign(prev_price = lambda x: x['price'].shift(1))
    .assign(daily_return = lambda x: (x['price'] - x['prev_price']) /
x['prev_price'])
    .assign(signal = lambda x: np.where(x['daily_return'] > 0, 'BUY',
'SELL'))
    .dropna() # 移除由shift()创建的第一行
)

print(df_final.head())
```

### 为什么这是传奇?

你可以像读英文食谱一样读代码。

*"取数据框, 分配前一日价格, 计算收益, 分配信号, 删除空值。"*

没有临时变量污染你的内存命名空间。

---

## 第6部分: 作业

你必须提交你的笔记本 (`.ipynb`)。

### 你的挑战 (交付物):

1. 计算盈利: 假设你投资了\$100。

2. 如果你的策略说'BUY' (1)，你持有该分钟的股票（你获得/损失%变化）。

3. 如果你的策略说'SELL' (0)，你持有现金（0%收益）。

#### 4. 向量化ROI计算：

- 创建列 `strategy_return = df['signal_shifted'] * df['pct_change']`。
- 计算累积收益：`(1 + strategy_return).cumprod()`。

#### 必需图表：

绘制"买入持有"表现 vs "你的策略"表现。

#### 评分标准：

- **性能：** 代码必须在5秒内运行。
- **向量化：** 如果我在你的策略计算块中找到 `for` 或 `iter` 这个词 → **0分**。
- **可视化：** 清晰的线图，显示你的算法是否打败了市场。（提示：在随机游走市场中，可能不会，但代码必须正确）。